Informe — Transferencias entre cuentas bancarias

Laboratorio Spring Boot con IA · Aplicaciones FullStack · Tecnológico de Antioquia · 2026
Estudiante: Felipe Arango · Ejercicio propuesto (entrega 30 de octubre de 2026)

1. Objetivo

Construir una aplicación completa para trasladar dinero entre cuentas de un banco que muestre el **histórico de transacciones** y, al generar el resultado de cada transacción, un **código QR** con los datos básicos de la misma codificados. Se siguieron los pasos del laboratorio (reglas → configuración → entidades → repositorio → servicio → controlador → CORS → frontend → documentación).

2. Stack tecnológico

Capa	Tecnología
Backend	Java 21 (compatible con 25), Spring Boot 4.1, Spring Web MVC, Spring Data JPA/Hibernate, Bean Validation
Base de datos	PostgreSQL 16 (H2 en memoria para pruebas y perfil `h2`)
QR	ZXing 3.5.4 (PNG de 300×300, UTF-8)
API docs	springdoc-openapi (Swagger UI)
Frontend	React 19, TypeScript, Vite, Axios
Herramientas	Maven, Git/GitHub, Claude Code (reglas en `.cursor/rules/springboot.mdc`)

3. Arquitectura

```
React (:3000) ──Axios──▶ Controller ──▶ Service ──▶ Repository ──▶ PostgreSQL (accounts, transactions)
                         @Valid/HTTP     reglas + QR    Spring Data JPA
```

Paquete `co.edu.tdea.banktransfer`: `controller`, `service`, `repository`, `entity`, `dto` (records), `exception` (jerarquía sealed + `GlobalExceptionHandler`) y `config` (CORS y datos de ejemplo).

Modelo de datos

Tabla	Columnas principales
accounts	id, account_number (único), owner_name, balance DECIMAL(19,2), created_at, version (@Version)
transactions	id, reference (UUID único), source_account, target_account, amount DECIMAL(19,2), description, status (COMPLETED | REJECTED), failure_reason, created_at

4. Reglas de negocio y decisiones de diseño

- Las cuentas deben existir (404), ser distintas y el monto mayor que cero (400), y el origen debe tener saldo suficiente (422).
- Dinero con `BigDecimal`; débito, crédito y registro en **una sola transacción** de base de datos.
- Bloqueo pesimista de las cuentas siempre en el mismo orden (evita *deadlocks* en transferencias cruzadas).
- Los intentos rechazados hacen rollback y se guardan aparte (`REQUIRES_NEW`) como `REJECTED` con su motivo, así quedan en el histórico.
- Errores como `ProblemDetail` (RFC 9457); mensajes en español.

5. Código QR

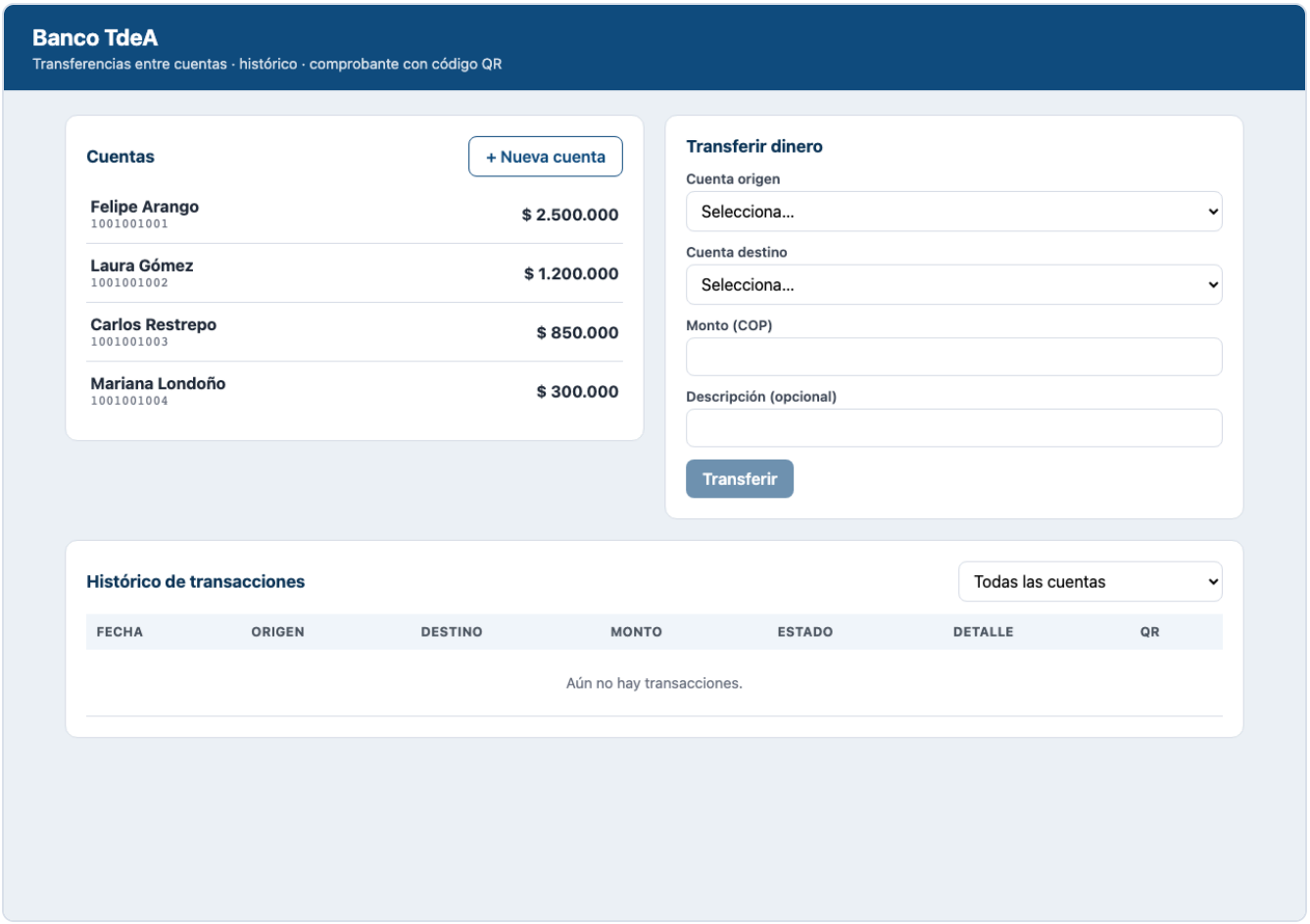
`QrCodeService` arma un texto con referencia, cuenta origen, cuenta destino, monto en COP, fecha y estado, y lo codifica con ZXing (corrección de errores M). La API lo devuelve como *data URI* en la respuesta de `POST /api/transferencias` y como PNG en `GET /api/transacciones/{referencia}/qr`. El frontend lo muestra en un modal descargable y desde el histórico (“Ver QR”).

6. API REST

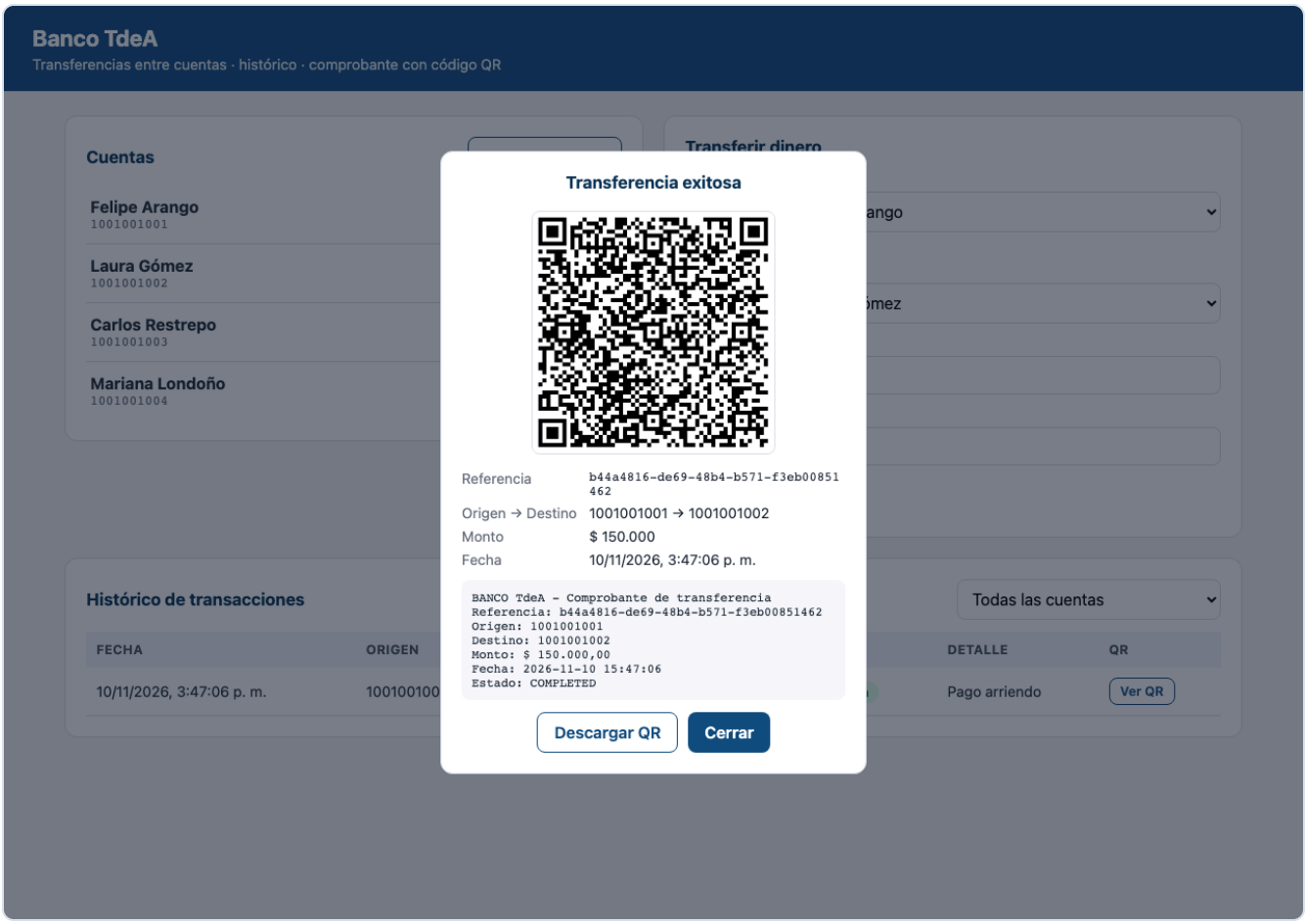
Método	Ruta	Descripción	Códigos
GET	/api/cuentas	Lista cuentas	200
GET	/api/cuentas/{numero}	Consulta una cuenta	200, 404

POST	/api/cuentas	Abre una cuenta	201, 400, 409
POST	/api/transferencias	Transfiere y devuelve QR	201, 400, 404, 422
GET	/api/transacciones?cuenta=	Histórico	200
GET	/api/transacciones/{referencia}/qr	PNG del QR	200, 400

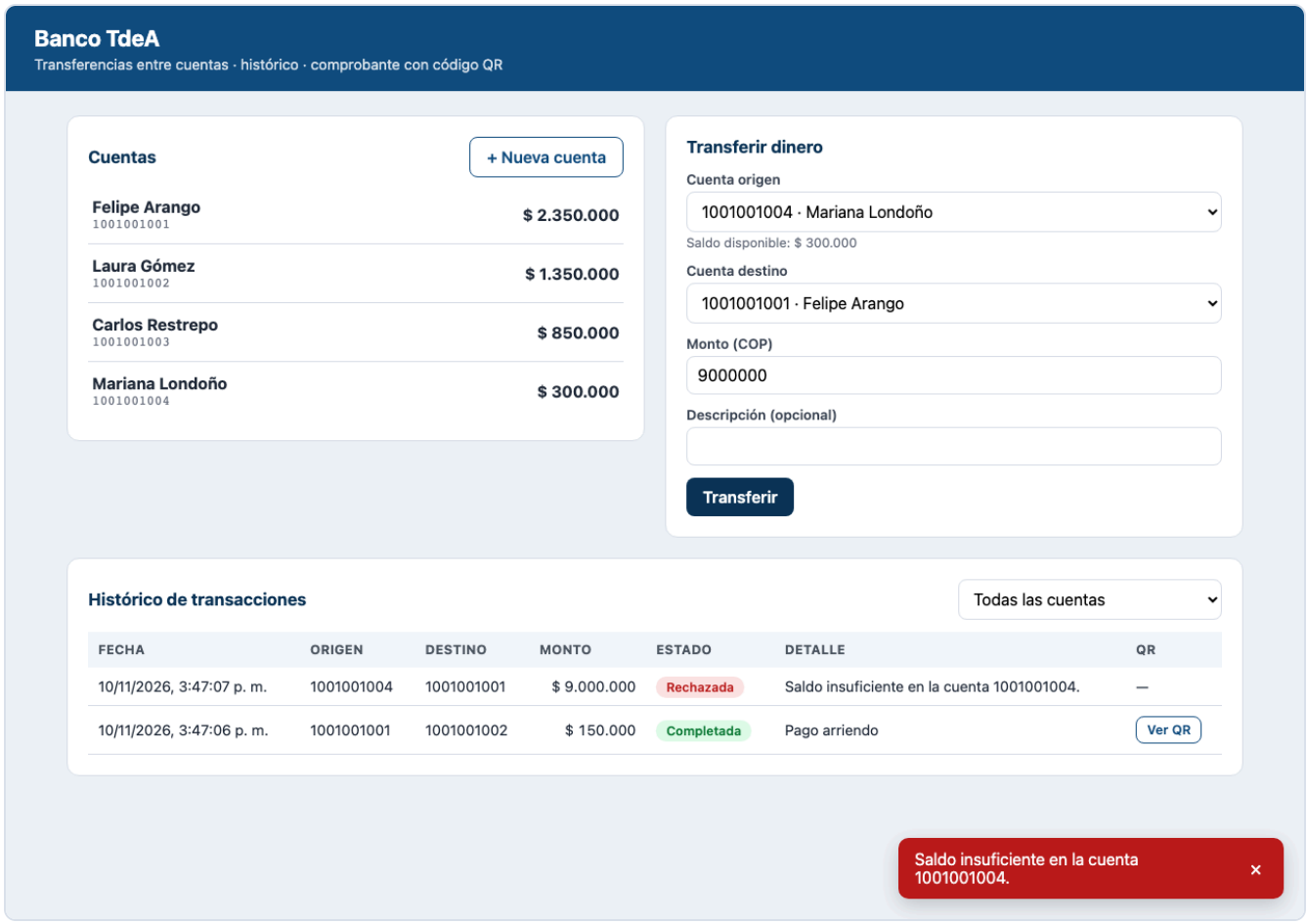
7. Evidencias (capturas)



Pantalla inicial: cuentas con saldo, formulario de transferencia e histórico.



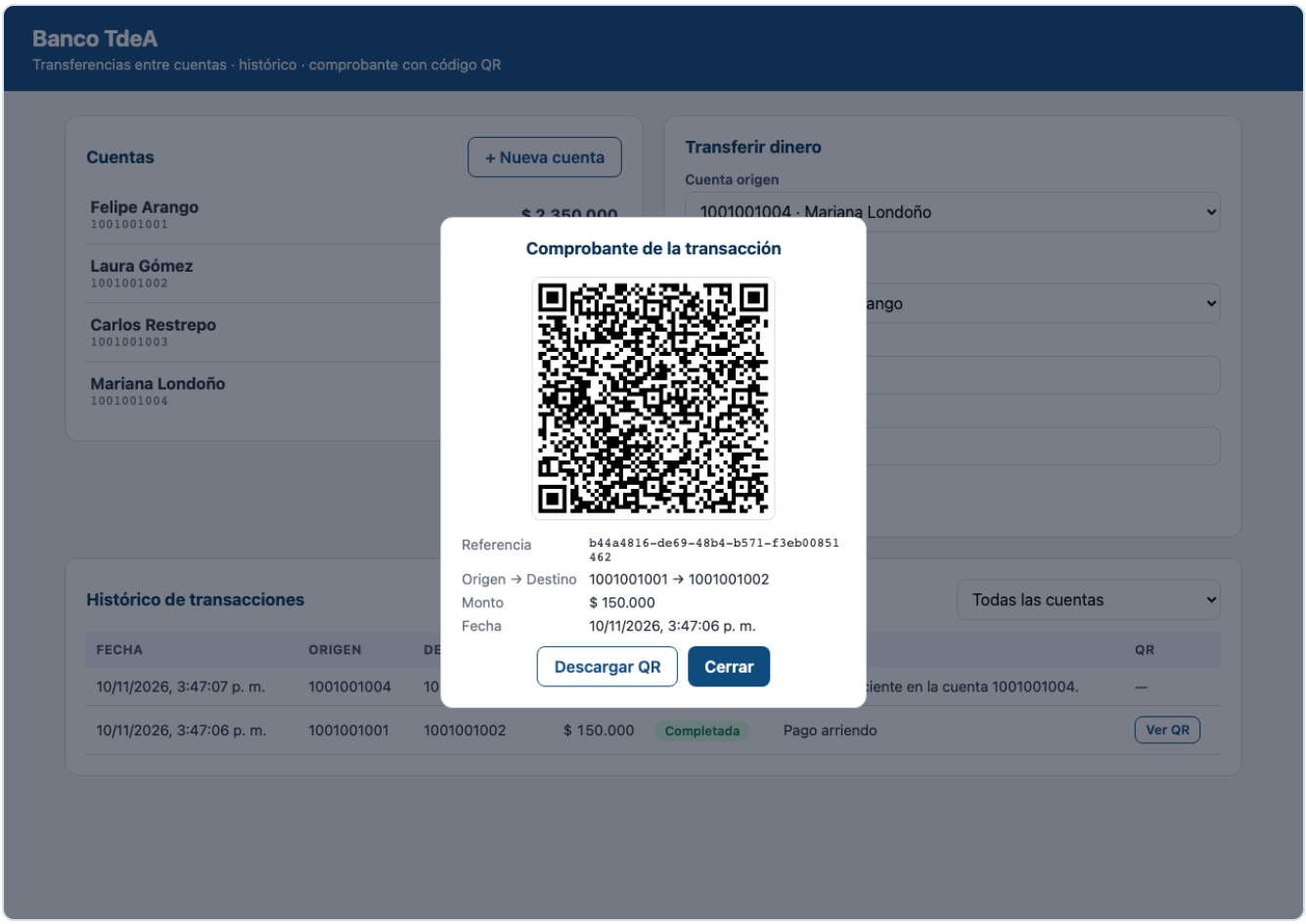
Transferencia exitosa con su código QR y los datos codificados.



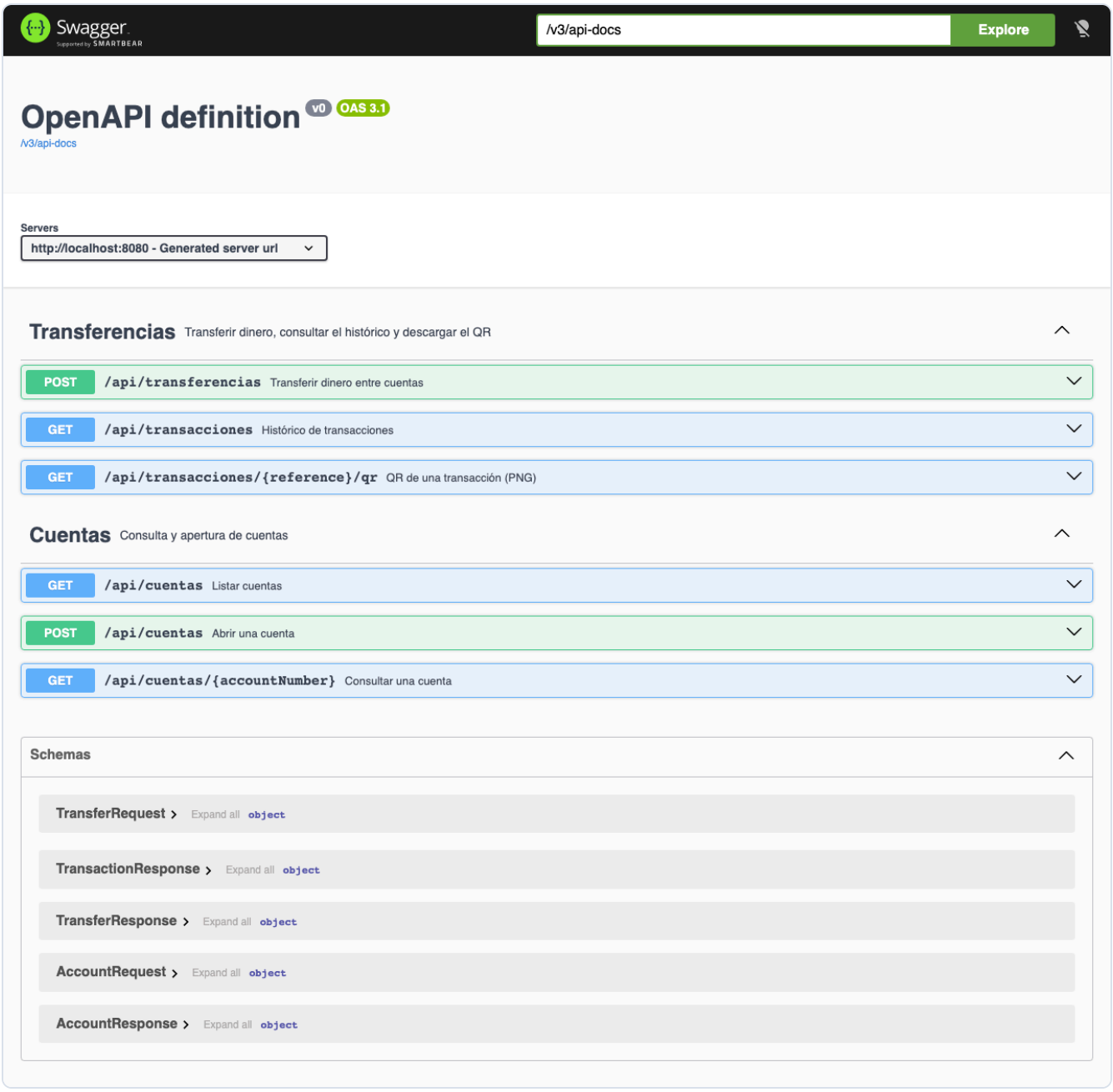
Transferencia rechazada por saldo insuficiente (toast con el detalle del backend).



Histórico con transacciones completadas y rechazadas.



QR de un movimiento abierto desde el histórico.



Swagger UI con los endpoints documentados.

Banco TdeA

Transferencias entre cuentas · histórico · comprobante con código QR

Cuentas

+ Nueva cuenta

Felipe Arango 1001001001	**$ 2.350.000**
Laura Gómez 1001001002	**$ 1.350.000**
Carlos Restrepo 1001001003	**$ 850.000**
Mariana Londoño 1001001004	**$ 300.000**

Transferir dinero

Cuenta origen

Selecciona...

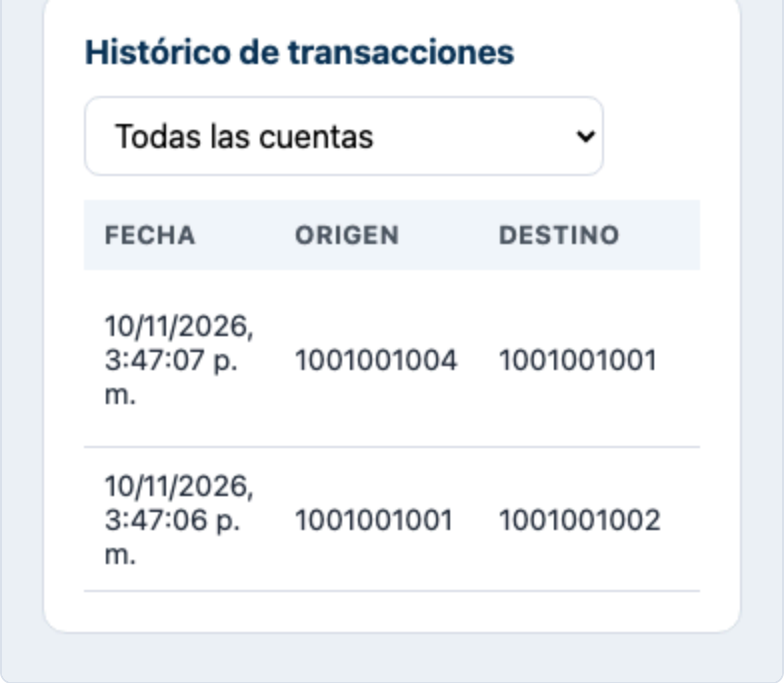
Cuenta destino

Selecciona...

Monto (COP)

Descripción (opcional)

Transferir



Vista adaptada a pantalla móvil.

8. Verificación

Prueba	Cómo se verifica
Pruebas de integración (5): éxito, saldo insuficiente, misma cuenta, cuenta inexistente, filtro del histórico	`cd backend && mvn test`
Frontend compila sin errores de tipos	`cd frontend && npm run build`
Datos guardados en PostgreSQL	`SELECT * FROM accounts; SELECT * FROM transactions;`

Consulta real con `psql` tras las transferencias de prueba (una exitosa y una rechazada):

```
$ psql -h localhost -p 5433 -U postgres -d banktdea   # contenedor banktdea-db
(PostgreSQL 16)
            List of relations
 Schema |     Name     | Type  |  Owner
--------+--------------+-------+----------
 public | accounts     | table | postgres
 public | transactions | table | postgres
(2 rows)

 account_number |   owner_name    |  balance
----------------+-----------------+------------
```

```
 1001001001     | Felipe Arango   | 2350000.00
 1001001002     | Laura Gómez     | 1350000.00
 1001001003     | Carlos Restrepo |  850000.00
 1001001004     | Mariana Londoño |  300000.00
(4 rows)

              reference               | source_account | target_account |   amount   |
status   |               failure_reason                |         created_at
--------------------------------------+----------------+----------------+------------+---
--------+---------------------------------------------+----------------------------
 b44a4816-de69-48b4-b571-f3eb00851462 | 1001001001     | 1001001002     |  150000.00 |
COMPLETED |                                             | 2026-11-10 15:47:06.894494
 668d0006-0e57-405e-924a-354134bdcb91 | 1001001004     | 1001001001     | 9000000.00 |
REJECTED  | Saldo insuficiente en la cuenta 1001001004. | 2026-11-10 15:47:07.653871
(2 rows)
```

9. Prompts utilizados

La secuencia completa de prompts está en `docs/PROMPTS.md`.

10. Conclusiones

El flujo del laboratorio (reglas → capas → frontend) se aplicó a un dominio con mayor criticidad: al manejar dinero fue necesario cuidar la precisión (`BigDecimal`), la atomicidad y la concurrencia, además de conservar los intentos fallidos en el histórico. La IA aceleró la generación de las capas, mientras que las reglas de negocio y las decisiones de diseño se revisaron y justificaron.